

Arrays and File Handling

No.	Exercise	File(s)	Marks
1	What Happened on This Trip?	trip.cpp	20
2	Plotting a Function in the Terminal	sine.cpp	20
3	Forest Fire Simulation	fire.cpp	35
4	Hiding a Secret Picture in a Grid of Numbers	hide.cpp, reveal.cpp	25
Total			100

Instructions

1. Submission

- Submit five separate .cpp files, named exactly as in the table above. Do not archive them into a single file unless the LMS requires it.
- Every program must compile with g++ without errors. A program that does not compile receives zero for that question.
- Include a comment block at the top of each file with your name, ERP ID, and section.
- Upload to the LMS before the deadline. Submissions sent by email are not accepted.

2. Code neatness:

use proper indentation, meaningful variable names, and comments explaining your logic, especially nested loops and file reading. Answer every *Think about it* box in a comment at the top of that exercise's code.

3. Allowed material:

every exercise can be solved using only what we have covered: variables, if, loops and nested loops, 1D and 2D arrays, rand()/srand(), <cmath>, and file streams (ifstream/ofstream), all inside main(). You do **not** need functions, pointers or recursion, and you must not use std::vector or std::sort. Use fixed-size arrays with a declared maximum (e.g. const int MAX_ROWS = 50;).

4. Files:

every program except sine.cpp reads its data from a file, and nothing that belongs in a file may be hardcoded. Always check that a file opened, print a clear message if it did not, and close every file you open. Input files are not always correct: each exercise lists what your program must reject, and deciding how to detect it is part of the task.

5. How the exercises are written:

they tell you *what* the program must do, not *how*. Working out the steps is part of the assignment. The provided input files are in student_files.zip.

6. Late policy:

submissions received after the deadline are penalized 10% per day, up to a maximum of three days. After 72 hours, the submission will not be graded. Extensions are granted only for documented medical or family emergencies.

7. Plagiarism:

the work must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) results in an automatic zero for the task.

Grading

Each question is marked on three things: correctness of output and logic, correct handling of the edge cases and the “must reject” inputs listed in the question, and code quality (meaningful variable names, consistent indentation, clear prompts, and your answers to the *Think about it* boxes).

	Ex. 1	Ex. 2	Ex. 3	Ex. 4
Correctness of output and logic	12	12	21	15
Edge cases / “must reject” handling	4	4	7	5
Code quality	4	4	7	5
Total	20	20	35	25

Assignment #2

Due: Oct 25 by 11:55pm

Exercise 1: What Happened on This Trip? Calculus from a Speed Log [20 marks]

A car's data logger does not record distance or acceleration. It only records the car's speed every few seconds, and the readings are *not* evenly spaced in time. An insurance company wants to know what the car actually did during a trip. Write a C++ program `trip.cpp` that reads the log and answers their questions.

Background: From calculus, distance is the area under the speed–time curve, and acceleration is the rate of change of speed. Here you have no formula for the curve, only a list of points. Between two readings you do not know exactly what the car did, so assume its speed changed at a steady rate. Turning that assumption into distance and acceleration is your job.

Input file trip.txt: the first line holds the number of readings N (at most 1000). Each of the next N lines holds a time in seconds and the speed at that time in m/s.

Your program must report:

- how long the trip lasted and how far the car travelled;
- the car's average speed over the whole trip;
- the hardest acceleration and the hardest braking, and when each happened;
- how long the car spent completely stopped.

Print the report on the screen and save it to `trip_report.txt`. The file must also contain a table showing, for every reading, how far the car had travelled up to that moment.

Must reject: a file in which time does not increase from one reading to the next, a speed is negative, or there are fewer readings than the first line promises. Say which reading is wrong.

Sample Input File: trip.txt

```
16
0 0
5 6
10 13
15 18
22 22
30 25
40 25
45 20
50 12
55 5
60 0
70 0
75 8
85 20
95 26
100 27
```

Sample Program Output

```
Screen output:
Trip duration:      100.00 s
Distance travelled: 1488.00 m
Average speed:     14.88 m/s
Hardest acceleration: 1.60 m/s^2 (between t = 70.00 s and t = 75.00 s)
Hardest braking:   -1.60 m/s^2 (between t = 45.00 s and t = 50.00 s)
Time fully stopped: 10.00 s
Full distance table written to trip_report.txt

trip_report.txt (table, first rows):
time(s)  speed(m/s)  distance(m)
0.00     0.00       0.00
5.00     6.00       15.00
10.00    13.00      62.50
15.00    18.00     140.00
22.00    22.00     280.00
...      ...       ...
```

Due: Oct 25 by 11:55pm

Exercise 2: *Plotting a Function in the Terminal* [20 marks]

Write a C++ program `sine.cpp` that plots the function $y = \sin(x)$ in the terminal window. The program should take as input the range of x values in radians over which to plot (for example, from -5 to 5). The output should be displayed on an 80-by-20 grid of characters, with the x-axis and y-axis clearly marked. The horizontal axis is represented by `-` at the vertical midpoint, and the vertical axis by `|` at the horizontal midpoint. The sine wave itself is represented by `*`.

Once the plot is complete, save it to `plot.txt` exactly as it appears on the screen, so that it can be opened in any text editor.

- To create the grid, use a 2D array of `char` with 20 rows and 80 columns, or a 1D array of 20 `std::strings` each of length 80. Initialize the grid with spaces, then fill in the axes and the sine wave points.
- Use the `sin()` function from `<cmath>`. It takes input in radians.
- Compute one point for *each column* of the grid. With 80 columns, the range is split into 79 equal steps, so column c (from 0 to 79) uses

$$x = x_{\min} + c \times \frac{x_{\max} - x_{\min}}{\text{grid_width} - 1}$$

For example, for $-2\pi \leq x \leq 2\pi$ the step is $4\pi/79 \approx 0.159$ radians: column 0 is $x = -2\pi$, column 1 is $x \approx -6.124$, and column 79 is $x = 2\pi$.

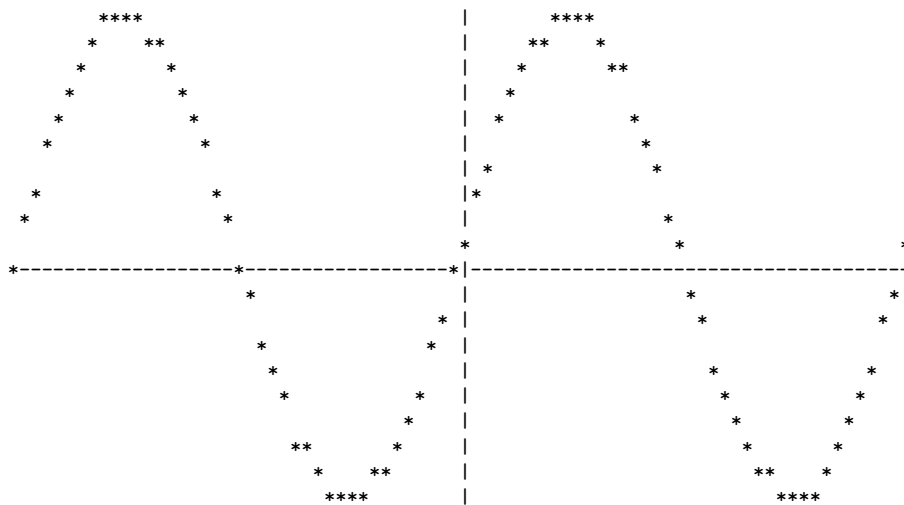
- To turn $y = \sin(x)$ into a row number, use

$$\text{grid_y} = \left(\frac{y - y_{\min}}{y_{\max} - y_{\min}} \right) \times (\text{grid_height} - 1)$$

rounded to the nearest whole number with `round()`, where $y_{\min} = -1$, $y_{\max} = 1$ and `grid.height` is 20. Row 0 is printed first, at the *top* of the screen, so a large y must go to a *small* row number.

Sample Output ($-2\pi \leq x \leq +2\pi$)

Enter xmin and xmax (radians): -6.2832 6.2832



Plot saved to plot.txt

Exercise 3: Forest Fire Simulation [35 marks]

The forest department wants to know how much of a forest a fire could destroy, and whether a river is enough to stop it. They have a map of the forest in a text file. Write a C++ program `fire.cpp` that reads the map, simulates the fire step by step, and writes a report.

The map. Each cell is one of:

Symbol	Meaning
T	Tree: can catch fire
1–9	Burning tree, with the fire's intensity (9 is the hottest)
A	Ash: burnt out, can never burn again
.	Empty ground: cannot burn
#	Water (a river or lake): cannot burn

Rules of the fire. In every step, all changes are decided from the map as it was at the start of that step.

- A burning cell tries to set fire to each of its four neighbours (up, down, left, right) that is a tree. Each attempt succeeds with probability p .
- **Wind:** the wind blows in one direction (N, S, E or W). Fire spreading in the direction of the wind succeeds with probability $2p$ (never more than 1); fire spreading against the wind succeeds with probability $p/2$.
- A tree that catches fire gets a random intensity between 1 and the intensity of the fire that lit it. If several fires reach the same tree in one step, the highest intensity wins.
- **Fuel:** each step, a burning cell's intensity goes up by 1 with probability g or down by 1 otherwise. Intensity never goes above 9.
- **Crown fire:** a cell burning at intensity 9 is so hot that it also spreads diagonally, to all eight neighbours.
- A burning cell turns to ash when its intensity drops to 0, or after it has burned for B steps, whichever comes first.
- The simulation runs for S steps, or stops early as soon as nothing is burning.

Input file forest.txt: the first line holds the number of rows and columns (at most 50×100). The second line holds S , p , g , B , the wind direction and a random seed. The map follows, one row per line. Use the seed with `srand()` so that the same file always gives the same fire.

Output: the initial map, the number of trees, burning cells and ash after every step, and the final map. Write the same information to `fire_report.txt`, ending with the percentage of the original forest (trees plus burning cells) that was destroyed.

Must reject: a missing file, a map that is too big, a row of the wrong length, or a symbol not in the table.

Due: Oct 25 by 11:55pm

[illegible]

```
Initial map:  
(same as the map in forest.txt)  
Step 1: Trees: 445, Burning: 5, Ash: 0  
Step 2: Trees: 437, Burning: 13, Ash: 0  
Step 3: Trees: 425, Burning: 24, Ash: 1  
...  
Step 24: Trees: 266, Burning: 3, Ash: 181  
Step 25: Trees: 266, Burning: 3, Ash: 181
```

```
Final map:  
AAAAAAAAAAAAAA#TTTTTTTTTTTTTTTTTTT  
AAAAAAAAAAAAAA#TTTTTTTT..TTTTTTTTTT  
AAAAAAAAAAAAAA#TTTTTT...TTTTTTTTTT  
AAAAAAAAAAAAA##TTTTTT..TTTTTTTTTT  
AAAAAA..AAAAAAA#TTTTTTTTTTTTTTTTTT  
AAAAA...AAAAAAA#TTTTTTTTTTTTTTTTTT  
AAAAAA..AAAAAAA#TTTTTTTTTTTTTTTTTT  
AAATAAAAAA#TTTTTTTTTTTTTTTTTTTTTT  
AAAAAAAAAAAAAA#TTTTTTTTTTTTTTTTTT  
AAAAAAAAAAAAAA#TTTTTTTTTTTTTTTTTT  
AAAAAAAAAAAAAA#TTTTTTTTTTTTTTTTTT  
211AAAAAAAAAAAAA#TTTTTTTTTTTTTTTTTT  
Forest destroyed: 181 of 450 (40.22%)
```

- Read the map one character at a time with `inFile >> ch;` into a 2D array. The `>>` operator skips the newlines for you.
- Storing each cell as an `int` (e.g. 0 for a tree, 1–9 for burning, negative numbers for ash, ground and water) makes the rules easier to write than storing characters.
- All changes in a step must be decided from the *old* map. Build the next step in a second 2D array and copy it back at the end of the step.
- A third 2D array can count how many steps each cell has been burning.
- `rand() / (RAND_MAX + 1.0)` gives a random `double` between 0 and 1; compare it with p .

- Change the wind to W and run again. Does the river still protect the east side? Why?
- Make a map where the river has a gap one cell wide. How often does the fire get across?
- Find roughly the smallest p at which the fire usually burns most of the forest.

Exercise 4: Hiding a Secret Picture in a Grid of Numbers.....[25 marks]

One way to send a secret is to scramble it so that only your friend can unscramble it (*cryptography*). Another is to hide it inside something so ordinary that nobody thinks to look (*steganography*). In this exercise you will hide a black-and-white picture inside a grayscale picture, where no human eye could spot it.

How a picture is stored: an image is a grid of tiny dots called *pixels*. In a grayscale image each pixel is a single number from 0 (black) to 255 (white). Two pixels with brightness 200 and 201 look identical to a person, but a computer can tell them apart perfectly. That is the whole trick.

The setting: a *cover* grid holds numbers from 0 to 255 (an ordinary grayscale picture). A *secret* grid of the same size holds only 0s (white) and 1s (black). You may change each cover number by **at most 1**, and every number must stay between 0 and 255. After hiding, every **odd** number in the grid means a secret **1** and every **even** number means a secret **0**.

(a) Hiding. Write `hide.cpp`, which reads `cover.txt` and `secret.txt` and writes the modified cover grid to `stego.txt`. All three files start with a line giving the number of rows and columns (each at most 50), followed by the grid. Print how many cover values had to change. Work out for yourself what to do with numbers at the edges of the allowed range.

(b) Revealing. Write `reveal.cpp`, which reads `stego.txt`, recovers the secret grid, writes it to `recovered.txt`, and draws it on the screen with `#` for 1 and a space for 0. Then compare it with `secret.txt` and report whether the secret came through perfectly, or how many cells differ.

Files provided: a 20×40 `cover.txt` and `secret.txt`. Do not open `secret.txt` before you run both programs: if your code is correct, `reveal.cpp` will draw the hidden picture for you.

Must reject: grids of different sizes, a secret containing anything other than 0 or 1, cover values outside 0–255, and files with fewer values than the first line promises.

Sample Input Files	
cover.txt 3 4 100 150 200 255 10 11 12 13 44 45 46 47	secret.txt 3 4 1 0 1 0 0 1 0 1 1 1 0 0
Sample Output Files	
stego.txt (created by hide.cpp) 3 4 101 150 201 254 10 11 12 13 45 45 46 48	recovered.txt (created by reveal.cpp) 3 4 1 0 1 0 0 1 0 1 1 1 0 0
Sample Screen Output	
hide.cpp prints: 5 of 12 values changed.	reveal.cpp prints: # # # # ## Secret recovered perfectly (0 of 12 cells differ).

Assignment #2

Due: Oct 25 by 11:55pm

Think about it (answer in a comment at the top of your code)

- Roughly what fraction of the cover values do you expect to change? Does the provided 20×40 example agree?
- Could someone who does not know the trick tell that `stego.txt` holds a secret? What might give it away?